24.4 浏览器服务器

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~190k tokens，输出 ~5k tokens（本节）

前三节我们依次解析了浏览器控制的三个层次：架构概览、CDP 低级层、Playwright 高级层。这一节分析最外层——**浏览器 HTTP 服务器**，它将所有能力封装为 RESTful API，供 Gateway 和 Agent 工具层调用。

24.4.1 浏览器 HTTP 服务器

服务器启动

```
// src/browser/server.ts — startBrowserControlServerFromConfig
export async function startBrowserControlServerFromConfig() {
  const cfg = loadConfig();
  const resolved = resolveBrowserConfig(cfg.browser, cfg);
  if (!resolved.enabled) return null;

  const app = express();
  app.use(express.json({ limit: "1mb" }));

  // 注册路由
  const ctx = createBrowserRouteContext({ getState: () => state });
  registerBrowserRoutes(app, ctx);

  // 绑定到 127.0.0.1（仅本地访问）
  const server = await app.listen(resolved.controlPort, "127.0.0.1");

  state = {
    server,
    port: resolved.controlPort,
    resolved,
    profiles: new Map(),
  };

  // 为 extension 模式的配置文件预启动 Relay 服务器
  for (const name of Object.keys(resolved.profiles)) {
    const profile = resolveProfile(resolved, name);
    if (profile?.driver === "extension") {
      await ensureChromeExtensionRelayServer({ cdpUrl: profile.cdpUrl });
    }
  }

  return state;
}
```

几个关键设计点：

- **绑定到 `127.0.0.1`** ——浏览器控制服务仅本地可访问，不暴露到网络

- **JSON 请求限制 1MB**——防止恶意大请求
- **profiles Map**——每个配置文件的运行时状态（浏览器进程、最后活跃标签页等）
- **Extension Relay 预启动**——如果配置文件使用 Chrome 扩展模式，在服务器启动时就启动 Relay，不等第一次操作

优雅关闭

```
export async function stopBrowserControlServer() {
  // 1. 停止所有配置文件的浏览器
  for (const name of Object.keys(current.resolved.profiles)) {
    await ctx.forProfile(name).stopRunningBrowser();
  }

  // 2. 关闭 HTTP 服务器
  await new Promise(resolve => current.server?.close(resolve));

  // 3. 关闭 Playwright 连接
  const mod = await import("./pw-ai.js");
  await mod.closePlaywrightBrowserConnection();
}
```

24.4.2 服务器上下文与标签页管理

配置文件上下文

`createBrowserRouteContext()` 为每个路由请求创建**配置文件作用域**的操作上下文：

```typescript
// src/browser/server-context.ts — ProfileContext 接口（推导）
interface ProfileContext {
  listTabs(): Promise<BrowserTab[]>;
  openTab(url: string): Promise<BrowserTab>;
  closeTab(targetId: string): Promise<void>;
  focusTab(targetId: string): Promise<void>;
  ensureBrowserRunning(): Promise<void>;
  stopRunningBrowser(): Promise<void>;
  getStatus(): ProfileStatus;
  getLastTargetId(): string | null;
  setLastTargetId(id: string): void;
}
```

```typescript
// src/browser/server-context.types.ts — BrowserTab 类型
type BrowserTab = {
  targetId: string;
  title?: string;
  url?: string;
  type?: string;
  wsUrl?: string;        // CDP WebSocket URL
};
```

配置文件运行时状态

```typescript
type ProfileRuntimeState = {
  profile: ResolvedBrowserProfile;
  running: RunningChrome | null;  // 当前运行的 Chrome 进程（如果有）
  lastTargetId: string | null;    // 最后操作的标签页
};
```

`lastTargetId` 的设计目的：当 Agent 不指定 `targetId` 时，默认使用最后操作的标签页。这符合人类的浏览习惯——"继续在当前标签页操作"。

标签页操作

标签页的打开支持两种方式：

```
// 本地浏览器：通过 CDP 创建
const { targetId } = await createTargetViaCdp({
  cdpUrl: profile.cdpUrl,
  url: targetUrl,
});

// 远程浏览器：通过 Playwright 创建
const page = await createPageViaPlaywright({
  cdpUrl: profile.cdpUrl,
  url: targetUrl,
});
```

标签页关闭类似——本地通过 CDP `/json/close/${targetId}`，远程通过 Playwright。

24.4.3 客户端动作层

`client-actions.ts` 是四个子模块的聚合：

```
// src/browser/client-actions.ts
export * from "./client-actions-core.js";     // 核心操作
export * from "./client-actions-observe.js"; // 观察操作
export * from "./client-actions-state.js";    // 状态操作
export * from "./client-actions-types.js";    // 类型定义
```

客户端动作层位于浏览器服务器路由和 Playwright 工具核心之间——它将 HTTP 请求参数转换为 Playwright 函数调用，并格式化响应。

核心动作（core）

```
click     → clickViaPlaywright
hover     → hoverViaPlaywright
type      → typeViaPlaywright
fill_form → fillFormViaPlaywright
select    → selectOptionViaPlaywright
press_key → pressKeyViaPlaywright
drag      → dragViaPlaywright
```

观察动作（observe）

```
snapshot    → snapshotAiViaPlaywright / snapshotRoleViaPlaywright
screenshot  → takeScreenshotViaPlaywright
console     → getConsoleMessagesViaPlaywright
network     → getNetworkRequestsViaPlaywright
errors      → getPageErrorsViaPlaywright
```

状态动作（state）

```
navigate    → navigateViaPlaywright
evaluate    → evaluateViaPlaywright
cookies     → cookiesGetViaPlaywright / cookiesSetViaPlaywright
storage     → storageGetViaPlaywright / storageSetViaPlaywright
resize      → resizeViewportViaPlaywright
```

24.4.4 Chrome 扩展中继

为什么需要扩展中继?

在 `extension` 驱动模式下，用户的 Chrome 不是由 OpenClaw 启动的——它没有 `--remote-debugging-port` 参数，无法直接通过 CDP 连接。Chrome Extension Relay 解决这个问题：

```
┌──────────────────┐          ┌──────────────────┐
│   OpenClaw       │          │   用户的 Chrome    │
│   (CDP Client)   │ ──WS──▶  │                  │
│                  │          │  Extension Relay  │
│  Relay Server    │ ◀──WS──  │  (Chrome 扩展)    │
│  (本地 HTTP+WS)  │          │                  │
└──────────────────┘          └──────────────────┘
```

> **衍生解释**：Chrome 扩展（Extension）运行在浏览器内部，拥有比普通网页更高的权限——可以访问 `chrome.debugger` API，这个 API 提供了与 CDP 等价的功能。OpenClaw 的中继架构利用这一点：扩展在浏览器内部执行 CDP 命令，然后通过 WebSocket 将结果转发给 OpenClaw 的 Relay Server。

中继架构

```
// src/browser/extension-relay.ts — 消息类型
type ExtensionForwardCommandMessage = {
  id: number;
  method: "forwardCDPCommand";
  params: { method: string; params?: unknown; sessionId?: string };
};

type ExtensionResponseMessage = {
  id: number;
  result?: unknown;
  error?: string;
};

type ExtensionForwardEventMessage = {
  method: "forwardCDPEvent";
  params: { method: string; params?: unknown; sessionId?: string };
};
```

通信流程：

1. OpenClaw 通过本地 Relay Server 连接到扩展的 WebSocket
2. 发送 `forwardCDPCommand` 消息（包含 CDP 方法和参数）
3. 扩展在浏览器内执行 `chrome.debugger.sendCommand()`
4. 扩展将结果通过 `ExtensionResponseMessage` 返回
5. 浏览器事件（如页面加载完成）通过 `forwardCDPEvent` 推送

认证

```
const RELAY_AUTH_HEADER = "x-openclaw-relay-token";
```

Relay Server 在启动时生成随机 token，存储在配置中。扩展连接时需要提供此 token。这防止其他程序冒充扩展连接到 Relay Server。

目标管理

Relay 内部维护了一个**已连接目标（Connected Target）**表：

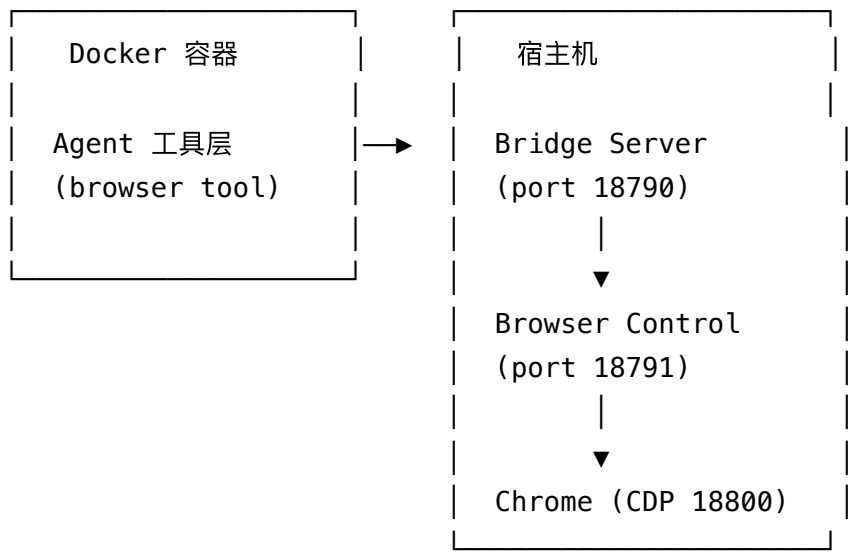
```
type ConnectedTarget = {
  sessionId: string;    // CDP session ID
  targetId: string;     // 标签页 ID
  targetInfo: TargetInfo; // 标签页信息
};
```

当扩展检测到 `Target.attachedToTarget` 事件时，将新目标加入表中；检测到 `Target.detachedFromTarget` 时移除。这样 OpenClaw 知道哪些标签页是可以操作的。

24.4.5 Bridge Server

`bridge-server.ts` 提供了一个额外的 HTTP 服务，充当 Agent 工具层和浏览器控制服务之间的**桥梁**。它在 Agent 运行的上下文中启动（可能在 Docker 容器内），将请求转发到宿主机上的浏览器控制服务。

这在沙箱模式下特别重要：



本节小结

1. **浏览器 HTTP 服务器**绑定到 `127.0.0.1` （仅本地），通过 Express 提供 RESTful API
2. **配置文件上下文**为每个浏览器实例提供隔离的操作接口——标签页列表、打开/关闭、启停浏览器

3. `lastTargetId` 记录最后操作的标签页，Agent 不指定时默认使用，符合人类浏览习惯
4. **客户端动作层**是路由和 Playwright 之间的适配器——核心/观察/状态三类动作覆盖所有浏览器操作
5. **Chrome 扩展中继**解决了"用户已有 Chrome"的问题——扩展在浏览器内执行 CDP 命令，通过 WebSocket 转发给 OpenClaw
6. **Bridge Server** 在沙箱模式下作为 Docker 容器和宿主机之间的转发器
7. 整个服务器栈：Agent Tool → Bridge (沙箱) → Browser Control Server → CDP/Playwright → Chrome